

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ

FACULTAD DE CIENCIAS E INGENIERÍA

ESPECIALIDAD DE INGENIERÍA INFORMÁTICA



Proyecto de Diseño y Desarrollo de Software
[1INF54-0982]

Estándar de Base de Datos y SQL

Equipo: 6F

Integrantes:

- Gabriel Angel Quispe Cerron
- Roly Alvaro Huacre Samaniego
- Amira Paola Rodriguez Almora
- Diego Alfredo Carmona Chumbimuni

2026-2

Lima, 15 de septiembre

Historial de versiones

Fecha	Versión	Descripción	Autor
15/09/2026	1.0	Se ha redactado la primera versión de los estándares de programación web frontend	Equipo 6F

Contenido

1. Introducción.....	4
2. Convenciones de nombres de tablas.....	4
3. Claves primarias, foráneas e índices.....	4
3.1. Claves primarias (PK).....	4
3.2. Claves foráneas (FK).....	4
3.3. Índices.....	5
3.4. Fechas y Zonas Horarias (Timestamps).....	5
3.5. Tipos de Datos Flexibles (JSONB).....	5
4. Reglas para consultas SQL eficientes.....	5
5. Estructuración de scripts de migración.....	6
6. Seeders para datos históricos.....	6
7. Seguridad y Transacciones.....	7
7.2. Seguridad a Nivel de Filas (Row-Level Security - RLS).....	7
7.3. Eliminación Lógica vs. Física (Soft Deletes).....	7
8. Referencias y recursos.....	7

1. Introducción

Este documento define los estándares para el diseño del esquema, la escritura de consultas SQL y la gestión de migraciones y datos semilla (seeders) en la base de datos PostgreSQL del sistema SysMile. Su objetivo es garantizar la consistencia del modelo de datos que soporta los módulos de pedidos, vehículos, conductores, rutas e historial de entregas.

2. Convenciones de nombres de tablas

- Nombrar las tablas en *snake_case* y en plural, reflejando el conjunto de entidades que almacenan (ej. *delivery_routes*, *packages*, *drivers*, *vehicles*, *route_stops*).
- Las tablas intermedias de relaciones muchos a muchos deben nombrarse combinando ambas entidades en singular separadas por guion bajo y en plural (ej. *route_vehicles*, *driver_licenses*).
- Las tablas de auditoría o historial deben llevar el sufijo *_history* o *_log* (ej. *route_status_history*).
- No usar prefijos redundantes como *tbl_* ni abreviaciones ambiguas.

3. Claves primarias, foráneas e índices

3.1. Claves primarias (PK)

- Toda tabla debe tener una clave primaria explícita, preferentemente un identificador UUID (columna *id* de tipo UUID) para facilitar la generación distribuida de identificadores en un entorno con múltiples instancias del backend.
- Usar *BIGSERIAL* únicamente en tablas de alto volumen puramente internas donde el orden secuencial aporte valor (ej. tablas de registros/logs).

3.2. Claves foráneas (FK)

- Toda clave foránea debe nombrarse con el nombre de la entidad referenciada en singular seguido del sufijo *_id* (ej. *route_id*, *driver_id*, *vehicle_id*).
- Definir explícitamente la acción de referencia (ON DELETE / ON UPDATE): usar *RESTRICT* por defecto para evitar borrados en cascada accidentales, y *CASCADE* solo en relaciones de composición estricta (ej. *route_stops* depende exclusivamente de *delivery_routes*).

SQL

```
ALTER TABLE route_stops
  ADD CONSTRAINT fk_route_stops_route
  FOREIGN KEY (route_id) REFERENCES delivery_routes(id)
```

```
ON DELETE CASCADE ;
```

3.3. Índices

- Crear índices sobre toda columna de clave foránea utilizada en uniones (joins) frecuentes.
- Crear índices compuestos para las combinaciones de filtros más usadas por el planificador (ej. *(driver_id, status)* en *delivery_routes* para consultar rutas activas de un conductor).
- Usar restricciones *UNIQUE* compuestas para invariantes de negocio, por ejemplo, evitar que un conductor tenga dos rutas activas simultáneas.
- Nombrar los índices con el prefijo *idx_* seguido de la tabla y las columnas (ej. *idx_delivery_routes_driver_status*).

3.4. Fechas y Zonas Horarias (Timestamps)

- Para garantizar la consistencia temporal en los eventos logísticos, es obligatorio usar exclusivamente el tipo *TIMESTAMP WITH TIME ZONE* (*timestamptz*) en lugar de *TIMESTAMP* para cualquier registro de fecha y hora. Esto asegura que PostgreSQL almacene todos los eventos en UTC absoluto, delegando la conversión y formateo a la zona horaria local a la capa del backend o frontend.

3.5. Tipos de Datos Flexibles (JSONB)

- El uso de columnas de tipo *JSONB* debe restringirse estrictamente a datos inherentemente no estructurados de solo lectura, almacenamiento de hiperparámetros o metadatos dinámicos de los algoritmos ALNS/IPSO, y bitácoras de respuestas de APIs externas.
- Cualquier dato con un esquema conocido que requiera ser filtrado, agrupado o indexado frecuentemente debe modelarse mediante normalización relacional tradicional.

4. Reglas para consultas SQL eficientes

- Evitar *SELECT **; especificar siempre las columnas necesarias, en particular en consultas embebidas en el cálculo de rutas donde se procesan grandes volúmenes de filas.
- Usar *EXPLAIN ANALYZE* antes de integrar una consulta compleja al código, verificando que los índices relevantes se estén utilizando (evitar *Seq Scan* en tablas grandes).
- Preferir uniones explícitas (*INNER JOIN*, *LEFT JOIN*) sobre la sintaxis implícita con coma en la cláusula *FROM*.

- Aplicar paginación basada en cursor (*keyset pagination*) para listados grandes, como el historial de entregas, en lugar de *OFFSET/LIMIT* sobre tablas de crecimiento continuo.
- Usar tipos *NUMERIC/DECIMAL* para distancias, costos y coordenadas que requieran precisión exacta; reservar *DOUBLE PRECISION* solo para cálculos intermedios no persistidos.
- Evitar el uso de funciones sobre columnas indexadas en la cláusula *WHERE* (ej. *LOWER(email) = ...*), salvo que exista un índice funcional equivalente.

5. Estructuración de scripts de migración

- Usar Flyway como herramienta de versionado de esquema, con los scripts ubicados en *src/main/resources/db/migration*.
- Nombrar los archivos de migración con el formato *V{n}__descripcion_en_snake_case.sql* (ej. *V12__add_priority_to_packages.sql*).
- Cada migración debe ser incremental y no debe modificarse una vez fusionada a la rama principal (*main*); cualquier corrección se realiza mediante una nueva migración.
- Las migraciones que afectan tablas con datos existentes deben incluir un plan de migración de datos (*UPDATE*) además del cambio de estructura (DDL), ejecutado dentro de la misma transacción cuando el motor lo permita.
- Evitar migraciones que bloqueen tablas de alto tráfico por tiempo prolongado; usar *CREATE INDEX CONCURRENTLY* para índices sobre tablas grandes en producción.

6. Seeders para datos históricos

- Los datos de prueba (*seeders*) históricos (usados para pruebas de los algoritmos ALNS/IPSO con volúmenes realistas) deben ubicarse en una carpeta separada *db/seeders*, distinta de las migraciones de esquema.
- Todo *seeder* debe ser idempotente: ejecutarlo múltiples veces no debe duplicar registros (usar *INSERT ... ON CONFLICT DO NOTHING* o truncar antes de insertar en entornos de prueba).
- Los *seeders* de datos históricos de gran volumen deben usar *COPY* o inserciones por lotes (*batch insert*) en lugar de sentencias *INSERT* individuales para minimizar el tiempo de carga.
- Documentar en un archivo *README.md* dentro de *db/seeders* el origen y la fecha de corte de los datos históricos cargados para asegurar la trazabilidad en los experimentos de optimización.

7. Seguridad y Transacciones

- 7.1. Control de Concurrencia y Aislamiento
- Definir el nivel de aislamiento *READ COMMITTED* por defecto; usar *SERIALIZABLE* únicamente en operaciones críticas de asignación concurrente de recursos (ej. asignación de un vehículo a una ruta).

7.2. Seguridad a Nivel de Filas (Row-Level Security - RLS)

- Para proteger la integridad y privacidad de la información en un entorno multiusuario, se deben definir políticas de *Row-Level Security* (RLS) nativas de PostgreSQL en las tablas críticas. Al igual que en ecosistemas modernos gestionados como Supabase, estas políticas garantizan a nivel de motor que un conductor, por ejemplo, solo pueda realizar un SELECT sobre las filas de *delivery_routes* que coincidan con su identificador, previniendo fugas de datos desde la capa de acceso.

7.3. Eliminación Lógica vs. Física (Soft Deletes)

- En un sistema altamente auditable, queda strictly prohibido el borrado físico (*DELETE*) de registros transaccionales (ej. pedidos, rutas) o datos maestros (ej. vehículos, conductores).
- Se debe implementar la "eliminación lógica" añadiendo una columna *deleted_at* (de tipo *timestampz*). El borrado físico se reservará únicamente para tablas intermedias efímeras o datos de sesión que carezcan de valor histórico.

8. Referencias y recursos

Documentación oficial de PostgreSQL: <https://www.postgresql.org/docs/>

Documentación de Flyway: <https://flywaydb.org/documentation/>

Guía de sintaxis SQL de PostgreSQL:
<https://www.postgresql.org/docs/current/sql-syntax-lexical.html>